

config 61-75

<https://gemini.google.com/app/b4e08fa861b91dc2>

Response: 1. 문제 원문

An internal MCP server requires a Kerberos-derived token that must be freshly minted for every connection, and no OAuth authorization server is involved. According to Anthropic documentation, which approach is most accurate for handling this authentication scheme?

A) Configure a static `headers` entry with the token value hardcoded, then rotate the config file manually whenever the token expires. B) Configure `headersHelper` to run a script that generates the token and writes the resulting header JSON to stdout on each connection. C) Configure the `oauth` block with `authServerMetadataUrl` pointed at the internal Kerberos realm so Claude Code discovers the flow automatically. D) Do not attempt to configure Kerberos directly in Claude Code MCP authentication; use an intermediary service that authenticates via Kerberos and then uses Anthropic-supported credentials such as API keys or OAuth tokens.

2. 구간별 직독직해 번역

QUESTION:

An internal MCP server 내부 MCP 서버가

requires a Kerberos-derived token Kerberos 기반 토큰을 요구합니다

that must be freshly minted 새로 발급되어야 하는

for every connection, 모든 연결마다,

and no OAuth authorization server is involved. 그리고 OAuth 인증 서버는 관여하지 않습니다.

According to Anthropic documentation, Anthropic 문서에 따르면,

which approach is most accurate 어떤 접근 방식이 가장 정확합니까?

for handling this authentication scheme? 이 인증 방식을 처리하기 위해?

OPTIONS:

Option A: Configure a static `headers` entry 정적 `headers` 항목을 설정하고

with the token value hardcoded, 토큰 값이 하드코딩된 채로,

then rotate the config file manually 그런 다음 설정 파일을 수동으로 교체합니다
whenever the token expires. 토큰이 만료될 때마다.

Option B: Configure `header sHelper` `header sHelper`를 설정하여
to run a script 스크립트를 실행하도록
that generates the token 토큰을 생성하고
and writes the resulting header JSON 결과 헤더 JSON을 작성하는
to stdout on each connection. 각 연결마다 표준 출력(stdout)으로.

Option C: Configure the `oauth block` `oauth` 블록을 설정하고
with `authServerMetadataUr l` `authServerMetadataUr l`과 함께
pointed at the internal Kerberos realm 내부 Kerberos 영역을 가리키는
so Claude Code discovers the flow automatically. Claude Code가 흐름을 자동으로 발견하도록.

Option D: Do not attempt to configure Kerberos directly Kerberos를 직접 설정하려고 시도하지
 마십시오

in Claude Code MCP authentication; Claude Code MCP 인증에서;
use an intermediary service 중계 서비스를 사용하십시오
that authenticates via Kerberos Kerberos를 통해 인증하는
and then uses Anthropic-supported credentials 그런 다음 Anthropic 지원 자격 증명을 사용하는
such as API keys or OAuth tokens. API 키나 OAuth 토큰 같은.

3. 정답 및 해설 (Answer & Explanation)

정답: B번: Configure `header sHelper` to run a script that generates the token and writes the resulting header JSON to stdout on each connection.

정답 및 해설: 핵심 개념: Dynamic Header Generation via `header sHelper` **in MCP (MCP 설정에**
서 `header sHelper` **를 이용한 동적 헤더 생성)** 정적 토큰이나 OAuth 외에, 매 연결마다 실시간으로 동적
 토큰을 생성하여 주입해야 하는 특수한 인증 스키마의 경우, Anthropic MCP 설정의 `header sHelper` 기능
 을 통해 스크립트를 실행하고 그 결과를 stdout으로 전달받아 처리할 수 있습니다.

문제 상황 분석:

- 내부 MCP 서버가 매 연결마다 새로 발급(minted)되어야 하는 Kerberos 파생 토큰을 요구함.
- 표준 OAuth 인증 서버가 관여하지 않으므로 일반적인 OAuth 흐름을 사용할 수 없음.
- 매번 만료되는 토큰을 수동 변경 없이 자동화하여 안전하게 전달할 수 있는 올바른 설정 방법을 찾아야 함.

B번이 정답인 이유:

- headersHelper 속성은 연결 시마다 스크립트를 실행하여 동적으로 인증 헤더(JSON 형식)를 생성하고 표준 출력(stdout)으로 내보낼 수 있도록 설계된 공식 메커니즘입니다.
- 매번 새로 발행되어야 하는 Kerberos 토큰의 특성에 완벽하게 부합합니다.

오답 분석:

- Option A (오답): 토큰이 만료될 때마다 설정을 수동으로 바꾸는 방식은 자동화 환경에 적합하지 않으며 비효율적입니다.
- Option C (오답): 문제에서 OAuth 인증 서버가 관여하지 않는다고 명시했으므로 oauth 블록을 사용하는 것은 전제 조건에 위배됩니다.
- Option D (오답): 불필요하게 별도의 중계 서비스를 도입할 필요 없이, Claude Code MCP의 headersHelper 기능을 통해 직접 해결할 수 있습니다.

User prompt: B가 오답인 이유: Although a script helper can supply dynamic headers, Anthropic documentation and bug reports indicate headersHelper runs when a transport is created and its output is cached for the transport lifetime, so it does not reliably re-run for every connection. Short-lived Kerberos-derived tokens can expire and cause 401 errors. D가 정답인 이유: Anthropic's documentation does not list a native Kerberos-derived authentication flow for MCP or the API. For integrating existing identity systems such as Kerberos, the recommended pattern is an intermediary service that authenticates the user or application with Kerberos, then securely obtains and uses Anthropic API keys or OAuth tokens, with a KMS for key handling. 위와 같았는데, 사실 확인해주

Response: 공식 앤스로픽(Anthropic) Claude Code 문서 및 가이드에 비추어 볼 때, 제시해주신 "B가 오답이고 D가 정답"이라는 주장은 사실이 아닙니다.

해당 문제의 정답은 B번(headersHelper)이 맞습니다. 세부적인 사실 관계는 다음과 같습니다.

- **공식 문서 기준 (headersHelper의 목적):** 앤스로픽의 공식 Claude Code MCP 인증 문서에서는 "Kerberos, 수명이 짧은 토큰(short-lived tokens), 내부 SSO 등 동적 헤더가 필요한 경우 .mcp.json에서 headersHelper를 사용하라"고 명시하고 있습니다.
- **B번이 정답인 이유:** headersHelper는 연결 시마다 스크립트를 실행하여 실시간으로 토큰을 생성하고, 그 결과로 나온 헤더 JSON을 표준 출력(stdout)으로 전달하여 만료되기 쉬운 토큰(Kerberos 등) 문제를 해결하도록 공식 지원되는 기능입니다.
- **D번이 오답인 이유:** 앤스로픽 문서에 Kerberos를 위한 네이티브 인증 플로우가 없다는 설명은 일부 맞을 수 있으나, 이를 해결하기 위해 불필요한 별도의 중계 서비스(Intermediary service) 구축을 강제하지 않으며, Claude Code 자체적으로 headersHelper를 통해 스크립트 기반의 동적 토큰 생성을 완벽히 지원하므로 D번은 틀린 설명입니다.

일부 커뮤니티나 이슈 트래커(예: 특정 버전에서의 캐싱 관련 버그나 제한 사항 논의)에서 제한적인 예외 상황이 언급되기도 하나, **아키텍처 설계 문항이나 공식 가이드 기준으로는 Kerberos 및 단기 토큰 처리에 headersHelper를 사용하는 것이 정석적인 답변입니다.**

Response: 1. 문제 원문

An architect is deciding how to scope a brand-new, still-unstable MCP server that only they should use, only within the one project they're actively prototyping in, without exposing it to teammates or to their other projects on the same machine. Which scope satisfies all of these constraints?

A) Project scope, since it stores the entry in .mcp.json and is intended to be shared with the whole team once committed B) User scope, since it stores the entry in ~/.claude.json and makes it available across every project that individual works on C) Local scope, since it stores the entry under that specific project's path in ~/.claude.json and stays private to the individual who added it D) Enterprise scope, since managed configuration is the only mechanism that restricts a server to a single named project

2. 구간별 직독직해 번역

QUESTION:

An architect is deciding 아키텍트가 결정하고 있습니다

how to scope 범위를 어떻게 설정할지

a brand-new, still-unstable MCP server 완전 새롭고 아직 불안정한 MCP 서버의

that only they should use, 오직 자신만 사용해야 하고,
only within the one project 오직 하나의 프로젝트 내에서만
they're actively prototyping in, 그가 활발히 프로토타이핑하고 있는,
without exposing it 노출하지 않고
to teammates 팀원들에게 혹은
or to their other projects 자신의 다른 프로젝트들에
on the same machine. 동일한 머신상의.

Which scope satisfies 어떤 범위가 만족합니까?
all of these constraints? 이러한 모든 제약 조건을?

OPTIONS:

Option A: Project scope, 프로젝트 범위,
since it stores the entry in .mcp.json 항목을 `.mcp.json`에 저장하고
and is intended to be shared 공유되도록 의도되었기 때문에
with the whole team once committed 일단 커밋되면 전체 팀과

Option B: User scope, 사용자 범위,
since it stores the entry in ~/.claude.json 항목을 `~/.claude.json`에 저장하고
and makes it available 사용 가능하게 만들기 때문에
across every project 모든 프로젝트에 걸쳐
that individual works on 그 개인이 작업하는

Option C: Local scope, 로컬 범위,
since it stores the entry 항목을 저장하기 때문에
under that specific project's path 해당 특정 프로젝트의 경로 아래에
in ~/.claude.json `~/.claude.json` 안의

and stays private 그리고 비공개로 유지되기 때문에
to the individual who added it 그것을 추가한 개인에게
Option D: Enterprise scope, 엔터프라이즈 범위,
since managed configuration 관리형 설정이
is the only mechanism 유일한 메커니즘이기 때문에
that restricts a server 서버를 제한하는
to a single named project 단일 명명된 프로젝트로

3. 정답 및 해설 (Answer & Explanation)

정답: C번: Local scope, since it stores the entry under that specific project's path in `~/.claude.json` and stays private to the individual who added it

정답 및 해설: 핵심 개념: MCP Configuration Scopes (MCP 설정 범위: Project, User, Local)

Claude Code 및 MCP 환경에서 설정의 가시성과 공유 범위를 제어하는 개념으로, 팀원 공유용(Project), 전역 사용용(User), 그리고 특정 프로젝트에만 국한되면서도 개인 설정으로 유지되어 저장소에 커밋되지 않는(Local) 범위를 구분합니다.

문제 상황 분석:

- 아키텍트 본인만 사용해야 하며, 팀원들에게 노출되면 안 됨.
- 현재 프로토타이핑 중인 특정 프로젝트 내에서만 동작해야 하며, 동일 머신상의 다른 프로젝트에는 영향을 주지 않아야 함.
- 저장소에 커밋되지 않고 개인(Local)의 환경에만 안전하게 격리되어야 함.

C번이 정답인 이유:

- 로컬 범위(Local scope)는 특정 프로젝트 경로와 연계하여 사용자 전역 설정 파일(`~/.claude.json`)에 저장되므로, 원격 저장소에 커밋되어 팀원들에게 공유되지 않습니다.
- 오직 해당 프로젝트를 작업하는 개인 개발자의 환경에서만 비공개로 유지되므로 문제에서 제시한 모든 제약 조건(개인 전용, 특정 프로젝트 한정, 팀 공유 안 함)을 완벽하게 충족합니다.

오답 분석:

- Option A (오답): 프로젝트 범위(Project scope)는 `.mcp.json`에 저장되어 커밋 시 팀 전체와 공유되므로 다른 팀원에게 노출되지 않아야 한다는 조건에 위배됩니다.

- Option B (오답): 사용자 범위(User scope)는 머신 내의 모든 프로젝트에서 전역적으로 사용 가능하므로, 특정 하나의 프로젝트로만 제한해야 하는 조건을 만족하지 못합니다.
- Option D (오답): 엔터프라이즈 범위(Enterprise scope)는 단일 명명 프로젝트 제한을 위한 메커니즘이 아니며, 개인 개발 중인 불안정한 서버를 격리하는 용도와 맞지 않습니다.

Response: 1. 문제 원문

A `run_report` MCP tool depends on a data warehouse that occasionally throttles requests with a 429 response. The tool author wants the agent to back off and retry automatically, but only up to a sensible limit, rather than retrying forever or giving up immediately. Which structured error response best supports this behavior?

A) `errorCategory`: "permission", `isRetryable`: false, and a description stating the report requires elevated warehouse access to proceed B) `errorCategory`: "validation", `isRetryable`: true, and a description asking the agent to reduce the report's date range before resubmitting the identical query C) No `errorCategory` or `isRetryable` field at all, relying on the phrase "try again later" in the text block to convey the retry semantics D) `errorCategory`: "transient", `isRetryable`: true, and a description noting the warehouse is rate-limiting requests, letting the agent apply its own bounded backoff strategy

2. 구간별 직독직해 번역

QUESTION:

A `run_report` MCP tool `run_report` MCP 도구가

depends on a data warehouse 데이터 웨어하우스에 의존합니다

that occasionally throttles requests 가끔 요청을 제한하는

with a 429 response. 429 응답으로.

The tool author wants 도구 작성자는 원합니다

the agent to back off 에이전트가 백오프(지연)하고

and retry automatically, 자동으로 재시도하기를,

but only up to a sensible limit, 그러나 합리적인 한도까지만,

rather than retrying forever 영원히 재시도하거나

or giving up immediately. 즉시 포기하는 것 대신에.

Which structured error response 어떤 구조화된 오류 응답이

best supports this behavior? 이러한 동작을 가장 잘 지원합니까?

OPTIONS:

Option A: `errorCategory: "permission", errorCategory: "permission",`

`isRetryable: false, isRetryable: false,`

and a description stating 그리고 설명하는 설명과 함께

the report requires elevated warehouse access 보고서가 향상된 웨어하우스 액세스를 요구한다는 것을

to proceed 진행하기 위해

Option B: `errorCategory: "validation", errorCategory: "validation",`

`isRetryable: true, isRetryable: true,`

and a description asking 그리고 요청하는 설명과 함께

the agent to reduce the report's date range 에이전트에게 보고서의 날짜 범위를 줄이도록

before resubmitting the identical query 동일한 쿼리를 다시 제출하기 전에

Option C: No errorCategory or isRetryable field at all, `errorCategory`나 `isRetryable` 필드가 전혀 없고,

relying on the phrase "try again later" "나중에 다시 시도하세요"라는 문구에 의존하는

in the text block 텍스트 블록 안의

to convey the retry semantics 재시도 의미를 전달하기 위해

Option D: `errorCategory: "transient", errorCategory: "transient",`

`isRetryable: true, isRetryable: true,`

and a description noting 그리고 언급하는 설명과 함께

the warehouse is rate-limiting requests, 웨어하우스가 요청 속도를 제한하고 있음을,

letting the agent apply 에이전트가 적용하도록 하면서

its own bounded backoff strategy 자신의 제한된 백오프 전략을

3. 정답 및 해설 (Answer & Explanation)

정답: D번: `errorCategory: "transient", isRetryable: true`, and a description noting the warehouse is rate-limiting requests, letting the agent apply its own bounded backoff strategy

정답 및 해설: 핵심 개념: Transient Error Handling and Rate Limiting in MCP Tools (MCP 도구 의 일시적 오류 처리 및 속도 제한) HTTP 429(Too Many Requests)와 같이 일시적으로 발생하는 속도 제한(Rate-limiting) 오류는 재시도가 가능한 성격이므로, 에이전트가 자체적인 백오프 전략을 적용할 수 있도록 `transient` 카테고리과 `isRetryable: true` 메타데이터를 제공해야 합니다.

문제 상황 분석:

- `run_report` 도구가 429 응답(속도 제한)을 가끔 발생시키는 데이터 웨어하우스와 연동되어 있음.
- 도구 작성자는 에이전트가 무한 재시도나 즉시 포기 대신, 합리적인 한도 내에서 지연(backoff)을 두고 자동 재시도하기를 바람.
- 에이전트가 이러한 의도를 정확히 파악하고 적절한 복구 조치를 취할 수 있는 구조화된 응답 형식을 선택해야 함.

D번이 정답인 이유:

- `errorCategory: "transient"`는 일시적인 장애 상태임을 명확히 나타냅니다.
- `isRetryable: true`는 에이전트에게 동일한 요청을 재시도할 수 있음을 알려줍니다.
- 설명에 속도 제한(rate-limiting) 상황을 명시함으로써 에이전트가 적절한 제한된 백오프(bounded backoff) 전략을 스스로 적용해 과도한 부하를 방지할 수 있게 지원합니다.

오답 분석:

- Option A (오답): `permission` 오류 및 `isRetryable: false` 설정은 권한 부족 문제를 뜻하므로 429 속도 제한 상황과 맞지 않으며 재시도를 차단합니다.
- Option B (오답): `validation` 오류 및 날짜 범위 축소 요구는 입력값 검증 실패 시나리오에 해당하며, 속도 제한(429)으로 인한 일시적 장애 상황에 부적합합니다.
- Option C (오답): 구조화된 필드(`errorCategory`, `isRetryable`) 없이 순수 텍스트("try again later")에만 의존하면 에이전트가 프로그래밍 방식으로 일관된 재시도 결정을 내리기 어렵습니다.

Response: 1. 문제 원문

An architect wants Claude Code to find every place in a codebase that throws an error containing the phrase "insufficient permissions", including messages built by concatenating a string literal across two lines with a plus sign. A single-line search pattern is not catching the split-line cases. What should Claude do?

A) Re-run Glob with the recursive ** pattern to search deeper into subdirectories where the missed messages might be hiding B) Re-run Grep with output mode count instead of content, since count mode is described as searching more thoroughly than content mode C) Switch to Read on every file in the repository so the full text of each file is visible instead of only matching lines D) Re-run Grep with multiline mode enabled so the pattern can match text that spans across the line boundary

2. 구간별 직독직해 번역**QUESTION:**

An architect wants 아키텍트가 원합니다

Claude Code to find Claude Code가 찾기를

every place in a codebase 코드베이스 내의 모든 장소를

that throws an error 오류를 발생시키는

containing the phrase "insufficient permissions", "insufficient permissions"라는 문구를 포함하는,

including messages built 메시지들을 포함하여

by concatenating a string literal 문자열 리터럴을 연결함으로써

across two lines 두 줄에 걸쳐

with a plus sign. 더하기 기호(+)로.

A single-line search pattern 단일 라인 검색 패턴은

is not catching 잡아내지 못하고 있습니다

the split-line cases. 줄이 나뉜 경우들을.

What should Claude do? Claude는 무엇을 해야 할까요?

OPTIONS:

Option A: Re-run Glob with the recursive ** pattern 재귀적 ** 패턴으로 Glob을 다시 실행합니다

to search deeper into subdirectories 더 깊은 하위 디렉토리로 검색하기 위해

where the missed messages might be hiding 놓친 메시지들이 숨어있을지도 모르는

Option B: Re-run Grep with output mode count 출력 모드를 count로 하여 Grep을 다시 실행합니다

instead of content, content 대신에,

since count mode is described count 모드가 설명되기 때문에

as searching more thoroughly 더 철저하게 검색하는 것으로

than content mode content 모드보다

Option C: Switch to Read on every file 모든 파일에 대해 Read로 전환합니다

in the repository 저장소 안의

so the full text of each file is visible 각 파일의 전체 텍스트가 보이도록

instead of only matching lines 일치하는 라인들뿐만 아니라

Option D: Re-run Grep with multiline mode enabled 멀티라인 모드가 활성화된 상태로 Grep을 다시 실행합니다

so the pattern can match text 패턴이 텍스트를 일치시킬 수 있도록

that spans across the line boundary 라인 경계를 가로질러 걸쳐 있는

3. 정답 및 해설 (Answer & Explanation)

정답: D번: Re-run Grep with multiline mode enabled so the pattern can match text that spans across the line boundary

정답 및 해설: 핵심 개념: Multiline Grep Search in Codebases (코드베이스에서의 멀티라인 Grep 검색) 코드 내에서 줄 바꿈(Line boundary)에 걸쳐 작성된 문자열이나 구문을 찾을 때는 단일 라인 검색만으로는 한계가 있으므로, 줄 경계를 넘나드는 패턴 매칭을 지원하는 멀티라인 모드(Multiline mode)를 활성화하여 Grep을 수행해야 합니다.

문제 상황 분석:

- "insufficient permissions" 문구를 포함하는 에러 발생 지점을 코드베이스 전체에서 찾고자 함.
- 일부 메시지는 더하기 기호(+)를 이용해 두 줄에 걸쳐 문자열 리터럴이 연결(Split-line)되어 있음.
- 일반적인 단일 라인 검색 패턴으로는 줄 바꿈으로 인해 끊어진 해당 케이스를 감지하지 못하고 있음.

D번이 정답인 이유:

- Grep 도구의 멀티라인 모드(Multiline mode)를 활성화하면 줄 바꿈 경계($\backslashn$)를 가로지르는 패턴 검색이 가능해집니다.
- 따라서 두 줄 이상에 걸쳐 나뉘어 작성된 문자열 연결 패턴도 정확하게 탐색할 수 있습니다.

오답 분석:

- **Option A (오답):** Glob은 파일 경로를 찾는 도구이며 내용 검색 능력이 없으므로 줄 바꿈된 문자열을 찾는 데 도움되지 않습니다.
- **Option B (오답):** Grep의 count 모드는 매칭된 '개수'만 세어주는 기능일 뿐, 검색 범위를 더 철저하게 넓히거나 멀티라인을 지원하지 않습니다.
- **Option C (오답):** 저장소의 모든 파일을 Read로 일일이 여는 것은 과도한 컨텍스트 토큰을 소모하며 매우 비효율적입니다.

Response: 1. 문제 원문

An architect wants Claude Code to find every place in a codebase that throws an error containing the phrase "insufficient permissions", including messages built by concatenating a string literal across two lines with a plus sign. A single-line search pattern is not catching the split-line cases. What should Claude do?

A) Re-run Glob with the recursive ** pattern to search deeper into subdirectories where the missed messages might be hiding
 B) Re-run Grep with output mode count instead of content, since count mode is described as searching more thoroughly than content mode
 C) Switch to Read on every file in the repository so the full text of each file is visible instead of only matching lines
 D) Re-run Grep with multiline mode enabled so the pattern can match text that spans across the line boundary

2. 구간별 직독직해 번역

QUESTION:

An architect wants 아키텍트가 원합니다

Claude Code to find Claude Code가 찾기를

every place in a codebase 코드베이스 내의 모든 장소를

that throws an error 오류를 발생시키는

containing the phrase "insufficient permissions", "insufficient permissions"라는 문구를 포함하는,

including messages built 메시지들을 포함하여

by concatenating a string literal 문자열 리터럴을 연결함으로써

across two lines 두 줄에 걸쳐

with a plus sign. 더하기 기호(+)로.

A single-line search pattern 단일 라인 검색 패턴은

is not catching 잡아내지 못하고 있습니다

the split-line cases. 줄이 나뉜 경우들을.

What should Claude do? Claude는 무엇을 해야 할까요?

OPTIONS:

Option A: Re-run Glob with the recursive ** pattern 재귀적 ** 패턴으로 Glob을 다시 실행합니다

to search deeper into subdirectories 더 깊은 하위 디렉토리로 검색하기 위해

where the missed messages might be hiding 놓친 메시지들이 숨어있을지도 모르는

Option B: Re-run Grep with output mode count 출력 모드를 count로 하여 Grep을 다시 실행합니다

instead of content, content 대신에,

since count mode is described count 모드가 설명되기 때문에

as searching more thoroughly 더 철저히 검색하는 것으로

than content mode content 모드보다

Option C: Switch to Read on every file 모든 파일에 대해 Read로 전환합니다

in the repository 저장소 안의

so the full text of each file is visible 각 파일의 전체 텍스트가 보이도록

instead of only matching lines 일치하는 라인들뿐만 아니라

Option D: Re-run Grep with multiline mode enabled 멀티라인 모드가 활성화된 상태로 Grep을 다시 실행합니다

so the pattern can match text 패턴이 텍스트를 일치시킬 수 있도록

that spans across the line boundary 라인 경계를 가로질러 걸쳐 있는

3. 정답 및 해설 (Answer & Explanation)

정답: D번: Re-run Grep with multiline mode enabled so the pattern can match text that spans across the line boundary

정답 및 해설: 핵심 개념: Multiline Grep Search in Codebases (코드베이스에서의 멀티라인 Grep 검색) 코드 내에서 줄 바꿈(Line boundary)에 걸쳐 작성된 문자열이나 구문을 찾을 때는 단일 라인 검색만으로는 한계가 있으므로, 줄 경계를 넘나드는 패턴 매칭을 지원하는 멀티라인 모드(Multiline mode)를 활성화하여 Grep을 수행해야 합니다.

문제 상황 분석:

- "insufficient permissions" 문구를 포함하는 에러 발생 지점을 코드베이스 전체에서 찾고자 함.
- 일부 메시지는 더하기 기호(+)를 이용해 두 줄에 걸쳐 문자열 리터럴이 연결(Split-line)되어 있음.
- 일반적인 단일 라인 검색 패턴으로는 줄 바꿈으로 인해 끊어진 해당 케이스를 감지하지 못하고 있음.

D번이 정답인 이유:

- Grep 도구의 멀티라인 모드(Multiline mode)를 활성화하면 줄 바꿈 경계($\backslash n$)를 가로지르는 패턴 검색이 가능해집니다.
- 따라서 두 줄 이상에 걸쳐 나뉘어 작성된 문자열 연결 패턴도 정확하게 탐색할 수 있습니다.

오답 분석:

- Option A (오답): Glob은 파일 경로를 찾는 도구이며 내용 검색 능력이 없으므로 줄 바꿈된 문자열을 찾는 데 도움되지 않습니다.
- Option B (오답): Grep의 count 모드는 매칭된 '개수'만 세어주는 기능일 뿐, 검색 범위를 더 철저하게 넓히거나 멀티라인을 지원하지 않습니다.
- Option C (오답): 저장소의 모든 파일을 Read로 일일이 여는 것은 과도한 컨텍스트 토큰을 소모하며 매우 비효율적입니다.

Response: 1. 문제 원문

A tool's description reads only "Get weather for a location." Users report the model sometimes garbles multi-word city names and invokes this tool when the user actually wants a historical climate report. What change would most directly reduce these failures?

A) Move all input validation into the tool's runtime error handler instead of describing constraints up front. B) Shorten the description further so the model spends less time interpreting it before invoking the tool. C) Expand the description to state the input format, include an example query, and note historical lookups are out of scope. D) Change the tool's name to a random unique string so it no longer semantically collides with any other tool at all.

2. 구간별 직독직해 번역

QUESTION:

A tool's description 도구의 설명이

reads only 오직 이렇게 읽힙니다

"Get weather for a location." "위치에 대한 날씨를 가져옵니다."라고.

Users report 사용자들이 보고합니다

the model sometimes garbles 모델이 때때로 뭉개거나 왜곡한다고

multi-word city names 여러 단어로 된 도시 이름을

and invokes this tool 그리고 이 도구를 호출한다고

when the user actually wants 사용자가 실제로 원할 때

a historical climate report. 역사적인 기후 보고서를.

What change 어떤 변경이

would most directly reduce these failures? 이러한 실패를 가장 직접적으로 줄일 수 있을까요?

OPTIONS:

Option A: Move all input validation 모든 입력 검증을 이동합니다

into the tool's runtime error handler 도구의 런타임 오류 핸들러 안으로

instead of describing constraints up front. 제약 조건을 사전에 설명하는 것 대신에.

Option B: Shorten the description further 설명을 더 단축합니다

so the model spends less time interpreting it 모델이 그것을 해석하는 데 더 적은 시간을 쓰도록

before invoking the tool. 도구를 호출하기 전에.

Option C: Expand the description 설명을 확장합니다

to state the input format, 입력 형식을 명시하고,

include an example query, 예시 쿼리를 포함하며,

and note historical lookups 역사적 조회가

are out of scope. 범위 외라는 점을 언급하기 위해.

Option D: Change the tool's name 도구의 이름을 변경합니다

to a random unique string 임의의 고유한 문자열로

so it no longer semantically collides 더 이상 의미론적으로 충돌하지 않도록

with any other tool at all. 다른 어떤 도구와도 전혀.

3. 정답 및 해설 (Answer & Explanation)

정답: C번: Expand the description to state the input format, include an example query, and note historical lookups are out of scope.

정답 및 해설: 핵심 개념: Tool Description Optimization & Disambiguation (도구 설명 최적화 및 명확화) LLM 에이전트 환경에서 도구 설명(Description)은 모델이 도구의 사용 목적과 제약 사항을 정확히

이해하고 올바른 인자를 전달하는 핵심적인 가이드 역할을 하므로, 입력 포맷, 예시, 제외 범위를 구체적으로 명시해야 오작동을 줄일 수 있습니다.

문제 상황 분석:

- 도구 설명이 "Get weather for a location."으로 너무 단순하여, 모델이 여러 단어의 도시 이름을 처리하는 방식(인자 포맷)을 자주 왜곡(garble)함.
- 사용자가 역사적 기후 보고서를 원할 때도 이 실시간 날씨 도구를 잘못 호출하는 의미적 혼동 (Semantic ambiguity)이 발생함.

C변이 정답인 이유:

- 설명(Description)을 확장하여 정확한 입력 형식을 정의하고 예시 쿼리를 제공하면, 모델이 여러 단어의 도시 이름을 올바른 형식으로 구성하여 인자 왜곡 문제를 방지할 수 있습니다.
- 또한 역사적 조치가 범위 외(out of scope)임을 명시함으로써, 사용자가 역사적 기후 보고서를 원할 때 이 도구가 잘못 호출되는 오남용을 직접적으로 차단할 수 있습니다.

오답 분석:

- **Option A (오답):** 사전 제약 조건을 설명하지 않고 런타임 오류 핸들러로 미루는 것은 모델의 사전 학습 및 인자 생성 오류를 예방하지 못하고 불필요한 런타임 실패를 유발합니다.
- **Option B (오답):** 이미 단순한 설명을 더 단축하면 모델이 도구의 용도와 제약을 파악할 단서가 부족해져 혼동과 실패가 가중됩니다.
- **Option D (오답):** 도구 이름을 의미 없는 무작위 문자열로 바꾸면 모델이 자연어 지시사항과 도구를 매칭하기 어려워져 사용성이 완전히 망가지므로 잘못된 접근입니다.

Response: 1. 문제 원문

Two tools, `analyze_content` (summarizes pasted text) and `analyze_document` (summarizes uploaded documents), share nearly identical one-line descriptions. Users report the model frequently calls `analyze_document` for pasted web article text instead of `analyze_content`. What is the most effective fix?

A) Rewrite each description to state the expected input type and add a note on when the other tool applies. B) Add a longer paragraph of promotional wording to `analyze_document` describing it as the more capable option. C) Remove `analyze_content` entirely from the tool

list so the model always falls back to `analyze_document` for every case. D) Increase the model's sampling temperature so it selects a more varied set of tools during response generation.

2. 구간별 직독직해 번역

QUESTION:

Two tools, 두 개의 도구가,

`analyze_content` (**summarizes pasted text**) `analyze_content` (붙여넣은 텍스트를 요약함)

and `analyze_document` (**summarizes uploaded documents**), 그리고 `analyze_document` (업로드된 문서를 요약함)가,

share nearly identical one-line descriptions. 거의 동일한 한 줄 설명을 공유합니다.

Users report 사용자들이 보고합니다

the model frequently calls `analyze_document` 모델이 빈번하게 `analyze_document`를 호출한다고

for pasted web article text 붙여넣은 웹 기사 텍스트에 대해

instead of `analyze_content`. `analyze_content` 대신에.

What is the most effective fix? 가장 효과적인 해결책은 무엇인가요?

OPTIONS:

Option A: Rewrite each description 각 설명을 다시 작성합니다

to state the expected input type 예상되는 입력 유형을 명시하고

and add a note 메모를 추가합니다

on when the other tool applies. 다른 도구가 적용되는 시점에 대한.

Option B: Add a longer paragraph 더 긴 단락을 추가합니다

of promotional wording 홍보성 문구의

to `analyze_document` `analyze_document`에

describing it 그것을 설명하면서

as the more capable option. 더 유능한 옵션으로.

Option C: Remove `analyze_content` entirely `analyze_content`를 완전히 제거합니다
from the tool list 도구 목록에서
so the model always falls back 모델이 항상 대체(fallback)하도록
to `analyze_document` `analyze_document`로
for every case. 모든 경우에.

Option D: Increase the model's sampling temperature 모델의 샘플링 온도를 높입니다
so it selects 선택하도록
a more varied set of tools 더 다양한 도구 세트를
during response generation. 응답 생성 동안.

3. 정답 및 해설 (Answer & Explanation)

정답: A번: Rewrite each description to state the expected input type and add a note on when the other tool applies.

정답 및 해설: 핵심 개념: Tool Description Disambiguation (도구 설명 명확화 및 구별) LLM 에이전트 환경에서 기능이 유사한 여러 도구가 존재할 때, 모델이 혼동하지 않도록 각 도구의 예상 입력 유형(Expected input type)과 상호 배타적인 사용 조건(When the other tool applies)을 명확하게 서술하여 도구 선택의 정확도를 높이는 최적화 기법.

문제 상황 분석:

- 두 도구(`analyze_content`, `analyze_document`)의 설명이 거의 유사하여 모델이 명확하게 구분하지 못함.
- 사용자가 붙여넣은 텍스트(Pasted text)를 입력했음에도 모델이 실수로 파일 업로드용 도구인 `analyze_document`를 빈번하게 호출함.
- 의미적 중첩(Semantic overlap)으로 인해 에이전트가 부적절한 도구를 선택하는 오작동이 발생함.

A번이 정답인 이유:

- 각 도구 설명에 구체적인 입력 형태(붙여넣은 텍스트 vs 업로드된 문서)를 명시하고, 어떤 상황에서 다른 도구를 써야 하는지 경계를 잡아줌으로써 모델이 문맥에 맞는 올바른 도구를 정확히 선택하도록 유도할 수 있습니다.

오답 분석:

- Option B (오답): `analyze_document`에 홍보성 문구를 추가하는 것은 도구의 기술적 기능이나 입력 제약을 명확히 하는 데 도움이 되지 않으며 오히려 모델에 혼란을 줍니다.
- Option C (오답): 고유한 역할이 있는 두 도구 중 하나를 강제로 삭제하는 것은 필수적인 기능을 상실하게 만듭니다.
- Option D (오답): 샘플링 온도를 높이면 응답의 무작위성만 커져 도구 선택의 정확성이 더욱 떨어지게 됩니다.

User prompt: 지금까지 확인한 문제를 분석했던 그대로 복사할 수 있는 markdown 블록으로 만들어줘. 문제는 61번부터 시작해서 75번까지 순서대로 번호매겨줘

Response: **61. 문제 원문**

A newly onboarded architect asks Claude Code to trace how a login request flows from the HTTP route handler through to the database call, in a codebase Claude has not explored yet. To build this understanding efficiently while keeping context usage low, what is the best incremental strategy?

A) Start by reading CLAUDE.md or AGENTS.md if they exist to gain high-level architecture context, then use Grep to locate the route handler and its imports, and read files incrementally along the call chain. B) Use Bash to run a full-text word count across the repository and read the files with the highest counts, on the assumption larger files hold core business logic. C) Use Read to open every file under the src directory up front, building a complete mental model of the whole codebase before looking for the login flow specifically. D) Use Glob to list every file in the repository sorted by modification time, then read the twenty most recently modified files on the assumption they relate to login.

62. 구간별 직독직해 번역

QUESTION:

A newly onboarded architect 새롭게 온보딩된 아키텍트가

asks Claude Code to trace Claude Code에 추적하도록 요청합니다

how a login request flows 로그인 요청이 어떻게 흐르는지

from the HTTP route handler HTTP 라우트 핸들러로부터

through to the database call, 데이터베이스 호출까지

in a codebase 코드베이스 내에서

Claude has not explored yet. Claude가 아직 탐색하지 않은

To build this understanding efficiently 이러한 이해를 효율적으로 구축하기 위해

while keeping context usage low, 컨텍스트 사용량을 낮게 유지하면서

what is the best incremental strategy? 가장 좋은 점진적 전략은 무엇인가요?

OPTIONS:

Option A: Start by reading CLAUDE.md or AGENTS.md CLAUDE.md 또는 AGENTS.md를 읽는 것으로 시작하여

if they exist 존재하는 경우

to gain high-level architecture context, 고위급 아키텍처 컨텍스트를 얻고

then use Grep to locate 그런 다음 Grep을 사용하여 찾고

the route handler and its imports, 라우트 핸들러와 해당 import들을

and read files incrementally 파일들을 점진적으로 읽는 것

along the call chain. 호출 체인을 따라

Option B: Use Bash to run a full-text word count Bash를 사용하여 전체 텍스트 단어 수를 실행하고

across the repository 저장소 전체에 걸쳐

and read the files 파일들을 읽는 것

with the highest counts, 가장 높은 단어 수를 가진

on the assumption 가정 하에

larger files hold core business logic. 더 큰 파일이 핵심 비즈니스 로직을 가지고 있다는

Option C: Use Read to open every file Read를 사용하여 모든 파일을 열고

under the src directory up front, src 디렉토리 아래의 파일들을 사전에 미리

building a complete mental model 완벽한 멘탈 모델을 구축하는 것

of the whole codebase 전체 코드베이스에 대한

before looking for the login flow specifically. 구체적으로 로그인 흐름을 찾기 전에

Option D: Use Glob to list every file Glob을 사용하여 모든 파일 목록을 조회하고

in the repository 저장소 내의

sorted by modification time, 수정 시간순으로 정렬하여

then read the twenty most recently modified files 최근에 수정된 20개 파일을 읽는 것

on the assumption 가정 하에

they relate to login. 그 파일들이 로그인과 관련이 있다는

63. 문제 원문

A subagent handling document translation calls a `translate_text` tool that fails with a connection timeout on the first attempt. The subagent's local retry policy allows up to 2 automatic retries for transient errors before escalating. The second retry also times out. What should the subagent do next?

A) Silently return a fabricated translation to the coordinator, using a cached fallback response from a prior successful call to avoid workflow interruption, despite the `translate_text` tool timeouts. B) Stop retrying and propagate the failure to the coordinator, noting the `errorCategory`, that timeouts persisted across the allowed local attempts, and what text remained untranslated. C) Continue retrying indefinitely at the subagent level, logging each attempt and resetting the retry count, ensuring that transient errors like timeouts are never escalated to the coordinator. D) Immediately reclassify the error as a permission failure and propagate it to the coordinator, including the permission flag and that repeated timeouts indicate an access-control issue, while aborting further retries.

64. 구간별 직독직해 번역

QUESTION:

A subagent 서브에이전트가

handling document translation 문서 번역을 처리하는

calls a `translate_text` tool `translate_text` 도구를 호출합니다

that fails with a connection timeout 연결 시간 초과로 실패하는

on the first attempt. 첫 번째 시도에서.

The subagent's local retry policy 서브에이전트의 로컬 재시도 정책은

allows up to 2 automatic retries 최대 2회의 자동 재시도를 허용합니다

for transient errors 일시적인 오류에 대해

before escalating. 상위(코디네이터)로 상격하기 전에.

The second retry also times out. 두 번째 재시도 역시 시간 초과됩니다.

What should the subagent do next? 서브에이전트는 다음에 무엇을 해야 합니까?

OPTIONS:

Option A: Silently return a fabricated translation 조용히 조작된 번역 결과를 반환합니다

to the coordinator, 코디네이터에게,

using a cached fallback response 캐시된 폴백 응답을 사용하여

from a prior successful call 이전의 성공적인 호출로부터 얻은

to avoid workflow interruption, 워크플로우 중단을 피하기 위해,

despite the translate_text tool timeouts. translate_text 도구 시간 초과에도 불구하고.

Option B: Stop retrying 재시도를 중지하고

and propagate the failure to the coordinator, 실패를 코디네이터에게 전파합니다,

noting the errorCategory, errorCategory를 기록하고,

that timeouts persisted 시간 초과가 지속되었음을 (기록하며)

across the allowed local attempts, 허용된 로컬 시도 횟수 동안,

and what text remained untranslated. 그리고 어떤 텍스트가 번역되지 않은 채로 남았는지를.

Option C: Continue retrying indefinitely 무한히 재시도를 계속합니다

at the subagent level, 서브에이전트 수준에서,

logging each attempt 각 시도를 로깅하고

and resetting the retry count, 재시도 횟수를 리셋하면서,
ensuring that transient errors 일시적인 오류가 (보장하도록)
like timeouts 시간 초과와 같은
are never escalated to the coordinator. 코디네이터에게 절대 상격되지 않도록.
Option D: Immediately reclassify the error 오류를 즉시 재분류합니다
as a permission failure 권한 실패로
and propagate it to the coordinator, 그리고 그것을 코디네이터에게 전파합니다,
including the permission flag 권한 플래그와
and that repeated timeouts indicate 반복된 시간 초과가 의미한다는 것을 포함하여
an access-control issue, 접근 제어 문제임을,
while aborting further retries. 추가 재시도를 중단하는 동안.

65. 문제 원문

A team is reviewing an MCP tool's error contract before launch. The current design returns `isError: true` with additional custom fields `errorCategory: "validation"` and `isRetryable: true` whenever the caller omits a required `customer_id` argument, on the reasoning that the agent might supply it correctly on a later attempt within the same conversation. What is the primary flaw in this retry guidance?

A) The `errorCategory` should be set to 'transient' instead of 'validation' because the error can be resolved later by the agent, implying a temporary condition. B) The flaw is that `isRetryable` should be false. A missing required argument is a validation error, which means the request is malformed. Retrying the same request will not succeed; the agent must correct the input before retrying. Setting `isRetryable` to true misleads the agent into thinking the same request can be retried as-is. C) The error contract includes fields that are not part of the MCP specification. MCP tool responses only define `isError` to indicate an error; there is no standard `errorCategory` or `isRetryable` field, so consumers may ignore these custom fields and miss the intended recovery guidance. D) The `isError` field should be set to false because the tool can still return a result when the `customer_id` is missing, making the invocation technically successful.

66. 구간별 직독직해 번역

QUESTION:

A team is reviewing 팀이 검토하고 있습니다

an MCP tool's error contract MCP 도구의 오류 계약을

before launch. 출시 전에.

The current design returns 현재 설계는 반환합니다

`isError: true` `isError: true`를

with additional custom fields 추가적인 커스텀 필드인

`errorCategory: "validation"` `errorCategory: "validation"`과

and `isRetryable: true` 그리고 `isRetryable: true`를

whenever the caller omits 호출자가 누락할 때마다

a required `customer_id` **argument,** 필수 `customer_id` 인자를,

on the reasoning 이유로

that the agent might supply it correctly 에이전트가 이를 올바르게 제공할 수도 있다는

on a later attempt 나중 시도에서

within the same conversation. 동일한 대화 내에서.

What is the primary flaw 주요 결함은 무엇인가요?

in this retry guidance? 이 재시도 지침의?

OPTIONS:

Option A: The `errorCategory` **should be set to** `'transient'` `errorCategory`는 `'transient'`로 설정되어야 합니다

instead of `'validation'` `'validation'` 대신에

because the error can be resolved later 오류가 나중에 해결될 수 있기 때문에

by the agent, 에이전트에 의해,

implying a temporary condition. 일시적인 상태를 암시하면서.

Option B: The flaw is that `isRetryable` should be false. 결함은 `isRetryable`이 false여야 한다는 것입니다.

A missing required argument 누락된 필수 인자는

is a validation error, 검증 오류이며,

which means the request is malformed. 이것은 요청이 잘못 형성되었음을 의미합니다.

Retrying the same request 동일한 요청을 재시도하는 것은

will not succeed; 성공하지 못할 것입니다;

the agent must correct the input 에이전트는 입력을 수정해야 합니다

before retrying. 재시도하기 전에.

Setting `isRetryable` to true `isRetryable`을 true로 설정하는 것은

misleads the agent 에이전트를 오도합니다

into thinking 생각하도록

the same request can be retried as-is. 동일한 요청이 그대로 재시도될 수 있다고.

Option C: The error contract includes fields 오류 계약은 포함합니다 필드들을

that are not part of the MCP specification. MCP 명세의 일부가 아닌.

MCP tool responses only define `isError` MCP 도구 응답은 오직 `isError`만 정의합니다

to indicate an error; 오류를 나타내기 위해;

there is no standard `errorCategory` 표준 `errorCategory`는 없습니다

or `isRetryable` field, 또는 `isRetryable` 필드가,

so consumers may ignore these custom fields 따라서 소비자들이 이 커스텀 필드들을 무시할 수 있고

and miss the intended recovery guidance. 의도된 복구 지침을 놓칠 수 있습니다.

Option D: The `isError` field should be set to false `isError` 필드는 false로 설정되어야 합니다

because the tool can still return a result 도구가 여전히 결과를 반환할 수 있기 때문에

when the `customer_id` is missing, `customer_id`가 누락되었을 때,
making the invocation technically successful. 호출을 기술적으로 성공한 것으로 만들면서.

67. 문제 원문

A shared `.mcp.json` points a stdio server's args at `${API_REGION:-us-east-1}`. On a machine where the `API_REGION` environment variable is unset, what value does Claude Code pass to the server?

A) An empty string, because Claude Code always expands an unset variable to blank rather than substituting the trailing default text
 B) The literal string `us-east-1`, because the `${VAR:-default}` syntax falls back to the default when the variable is not set
 C) A parse failure, because Claude Code requires every referenced environment variable to be set even when a default is supplied
 D) The literal text `${API_REGION:-us-east-1}`, because default-value expansion only applies inside the `env` block and not in `args`

68. 구간별 직독직해 번역

QUESTION:

A shared `.mcp.json` 공유된 `.mcp.json`이

points a stdio server's args stdio 서버의 args를 가리킵니다

at `${API_REGION:-us-east-1}`. `${API_REGION:-us-east-1}`로.

On a machine 머신에서

where the `API_REGION` environment variable is unset, `API_REGION` 환경 변수가 설정되어 있지 않은

what value does Claude Code pass 어떤 값을 Claude Code가 전달합니까?

to the server? 서버에?

OPTIONS:

Option A: An empty string, 빈 문자열,

because Claude Code always expands Claude Code가 항상 확장하기 때문에

an unset variable to blank 설정되지 않은 변수를 빈 값으로

rather than substituting 대체하기보다는

the trailing default text 뒤에 붙은 기본값 텍스트를

Option B: The literal string `us-east-1`, 리터럴 문자열 `us-east-1`,

because the `${VAR:-default}` **syntax** `${VAR:-default}` 문법이

falls back to the default 기본값으로 대체되기 때문에

when the variable is not set 변수가 설정되지 않았을 때

Option C: A parse failure, 구문 분석 실패,

because Claude Code requires Claude Code가 요구하기 때문에

every referenced environment variable to be set 참조된 모든 환경 변수가 설정되어 있기를

even when a default is supplied 기본값이 제공될 때조차도

Option D: The literal text `${API_REGION:-us-east-1}`, 리터럴 텍스트 `${API_REGION:-us-east-1}`,

because default-value expansion 기본값 확장이

only applies inside the `env` **block** `env` 블록 내부에만 적용되고

and not in `args`. `args`에는 적용되지 않기 때문에

69. 문제 원문

A coordinator agent delegates a three-step data migration to a subagent: extract, transform, and load, but the load step fails twice on a database connection reset, a known transient condition, before finally succeeding on the third attempt inside the subagent's own execution. What should the subagent report back to the coordinator?

A) An escalation asking the coordinator to obtain new database credentials, since two consecutive connection resets indicate the credentials have expired B) An `isError: true` result describing both connection resets in detail, so the coordinator can decide independently whether the migration should be retried C) A partial-results payload listing only the extract and transform steps as done, omitting the load step entirely since it initially failed twice D) A success result summarizing the completed migration, since the transient failures were resolved locally and never needed to surface above the subagent

70. 구간별 직독직해 번역

QUESTION:

A coordinator agent 코디네이터 에이전트가

delegates a three-step data migration 3단계 데이터 마이그레이션을 위임합니다

to a subagent: 서브에이전트에게:

extract, transform, and load, 추출, 변환, 그리고 적재,

but the load step fails twice 그러나 적재 단계가 두 번 실패합니다

on a database connection reset, 데이터베이스 연결 초기화로 인해,

a known transient condition, 알려진 일시적 상태인,

before finally succeeding 최종적으로 성공하기 전에

on the third attempt 세 번째 시도에서

inside the subagent's own execution. 서브에이전트 자체의 실행 내부에서.

What should the subagent report back 서브에이전트는 무엇을 보고해야 합니까?

to the coordinator? 코디네이터에게?

OPTIONS:

Option A: An escalation asking the coordinator 코디네이터에게 요청하는 에스컬레이션을

to obtain new database credentials, 새로운 데이터베이스 자격 증명을 획득하도록,

since two consecutive connection resets 두 번의 연속된 연결 초기화가 나타내므로

indicate the credentials have expired 자격 증명만료되었음을

Option B: An `isError: true` result `isError: true` 결과를

describing both connection resets in detail, 두 번의 연결 초기화를 상세히 설명하는,

so the coordinator can decide independently 코디네이터가 독립적으로 결정할 수 있도록

whether the migration should be retried 마이그레이션을 재시도해야 하는지를

Option C: A partial-results payload 부분 결과 페이로드를

listing only the extract and transform steps as done, 완료된 추출 및 변환 단계만 나열하는,
omitting the load step entirely 적재 단계를 완전히 누락한 채
since it initially failed twice 처음에 두 번 실패했기 때문에
Option D: A success result 성공 결과를
summarizing the completed migration, 완료된 마이그레이션을 요약하는,
since the transient failures were resolved locally 일시적 실패가 로컬에서 해결되었고
and never needed to surface above the subagent 서브에이전트 위로 표면화될 필요가 전혀 없었
 기 때문에

71. 문제 원문

A teammate manually edited a shared config file in a separate editor after Claude Code had already read it earlier in the session. Claude now attempts an Edit call against that file using a string it saw during its earlier read, and the call fails. What is the most likely reason, and the correct recovery step?

A) The `old_string` must have contained an unescaped regex metacharacter, so Claude should escape every character in `old_string` and retry the identical call B) Edit failed because the file was read more than one tool call ago, so Claude should discard the file and recreate it from scratch using Write C) The file changed on disk since Claude's earlier read, so the read-before-edit check fails; Claude should re-read it and retry Edit against updated text D) Edit calls always fail on the second attempt within a session, so Claude should switch permanently to Bash-based text replacement for the rest of the session

72. 구간별 직독직해 번역

QUESTION:

A teammate manually edited 동료가 수동으로 편집했습니다
a shared config file 공유 설정 파일을
in a separate editor 별도의 편집기에서
after Claude Code had already read it Claude Code가 이미 그것을 읽은 후에
earlier in the session. 세션 초기에.

Claude now attempts an Edit call Claude는 이제 Edit 호출을 시도합니다
against that file 해당 파일에 대해
using a string 문자열을 사용하여
it saw during its earlier read, 이전 읽기 동안 보았던,
and the call fails. 그리고 호출이 실패합니다.

What is the most likely reason, 가장 유력한 원인과
and the correct recovery step? 올바른 복구 단계는 무엇인가요?

OPTIONS:

Option A: The old_string must have contained old_string이 포함하고 있었음에 틀림없습니다
an unescaped regex metacharacter, 이스케이프되지 않은 정규식 메타문자를,
so Claude should escape 따라서 Claude는 이스케이프해야 합니다
every character in old_string old_string의 모든 문자를
and retry the identical call 그리고 동일한 호출을 재시도해야 합니다

Option B: Edit failed Edit이 실패했습니다
because the file was read 파일이 읽혔기 때문에
more than one tool call ago, 한 번의 도구 호출보다 더 전에,
so Claude should discard the file 따라서 Claude는 파일을 폐기하고
and recreate it from scratch 처음부터 다시 생성해야 합니다
using Write Write를 사용하여

Option C: The file changed on disk 파일이 디스크상에서 변경되었습니다
since Claude's earlier read, Claude의 이전 읽기 이후로,
so the read-before-edit check fails; 따라서 편집 전 읽기 확인(검증)이 실패합니다;
Claude should re-read it Claude는 그것을 다시 읽어야 하고

and retry Edit Edit을 재시도해야 합니다

against updated text 업데이트된 텍스트를 대상으로

Option D: Edit calls always fail Edit 호출은 항상 실패합니다

on the second attempt 두 번째 시도에서

within a session, 세션 내에서,

so Claude should switch permanently 따라서 Claude는 영구적으로 전환해야 합니다

to Bash-based text replacement Bash 기반 텍스트 교체로

for the rest of the session 나머지 세션 동안

73. 문제 원문

An architect is rolling out a GitHub MCP server for the whole engineering team. Every teammate has their own GitHub personal access token, and the config must be checked into the repo without ever committing a real secret. How should the architect configure this?

A) Add the server with project scope in `.mcp.json`, and set the header to `Authorization: Bearer ${GITHUB_TOKEN}` so each teammate's environment supplies the value at connection B) Add the server with local scope, then have every teammate individually edit their own copy of `.mcp.json` to insert their personal token in place of a placeholder C) Add the server with project scope, then add `.mcp.json` to `.gitignore` so the checked-in repository never actually contains the shared server configuration file D) Add the server with user scope in `~/.claude.json`, and paste each teammate's literal token value into the shared header field before committing that file to the repo

74. 구간별 직독직해 번역

QUESTION:

An architect is rolling out 아키텍트가 배포하고 있습니다

a GitHub MCP server GitHub MCP 서버를

for the whole engineering team. 전체 엔지니어링 팀을 위해.

Every teammate has 모든 팀원이 가지고 있습니다

their own GitHub personal access token, 자신만의 GitHub 개인 액세스 토큰을,

and the config must be checked into the repo 그리고 설정은 저장소에 체크인되어야 합니다
without ever committing a real secret. 실제 시크릿을 전혀 커밋하지 않고서.

How should the architect configure this? 아키텍트는 이것을 어떻게 설정해야 할까요?

OPTIONS:

Option A: Add the server with project scope 프로젝트 범위로 서버를 추가하고

in `.mcp.json`, `.mcp.json`에,

and set the header to `Authorization: Bearer ${GITHUB_TOKEN}` 헤더를 `Authorization: Bearer ${GITHUB_TOKEN}`으로 설정합니다

so each teammate's environment 각 팀원의 환경이

supplies the value at connection 연결 시점에 값을 제공하도록

Option B: Add the server with local scope, 로컬 범위로 서버를 추가하고,

then have every teammate individually edit 그런 다음 모든 팀원이 개별적으로 편집하도록 합니다

their own copy of `.mcp.json` 자신만의 `.mcp.json` 사본을

to insert their personal token 개인 토큰을 삽입하기 위해

in place of a placeholder 플레이스홀더 대신에

Option C: Add the server with project scope, 프로젝트 범위로 서버를 추가하고,

then add `.mcp.json` **to** `.gitignore` 그런 다음 `.mcp.json`을 `.gitignore`에 추가합니다

so the checked-in repository 체크인된 저장소가

never actually contains 실제로 포함하지 않도록

the shared server configuration file 공유 서버 설정 파일을

Option D: Add the server with user scope 사용자 범위로 서버를 추가하고

in `~/.claude.json`, `~/.claude.json` 안에서,

and paste each teammate's literal token value 각 팀원의 리터럴 토큰 값을 붙여넣습니다

into the shared header field 공유 헤더 필드에

before committing that file to the repo 해당 파일을 저장소에 커밋하기 전에

75. 문제 원문

A platform team is designing error responses for a fleet of internal MCP tools. One engineer proposes that every tool failure, regardless of cause, return the same generic text "Operation failed" with `isError: true`, arguing this keeps the interface simple for tool authors. What is the strongest architectural objection to this proposal?

A) Returning a constant error string for every failure adds metadata overhead that pushes the total block size beyond the MCP protocol's maximum content length, so the server rejects the tool result as non-compliant. B) A uniform generic message gives the agent no basis for choosing among retrying, adjusting input, or escalating, so it cannot make an appropriate recovery decision for each failure. C) Uniform error text prevents the server from ever setting `isError:true` because the MCP specification requires a unique diagnostic string to accompany the flag for each failure, so the tool cannot activate the error state. D) The MCP specification requires every `isError:true` result to include a machine-parseable stack trace, so a generic text response without that structured data violates the protocol and is rejected by the platform.

User prompt: 지금까지 확인한 문제들을 모아서 복사할 수 있는 *markdown* 블록으로 만들어줘. 각 문제 번호는 H1 레벨로, 그 안에 문제원문, 구간별 직독직해 번역, 정답과 해설을 모두 포함해

Response: 61번 문제

1. 문제 원문

A newly onboarded architect asks Claude Code to trace how a login request flows from the HTTP route handler through to the database call, in a codebase Claude has not explored yet. To build this understanding efficiently while keeping context usage low, what is the best incremental strategy?

A) Start by reading CLAUDE.md or AGENTS.md if they exist to gain high-level architecture context, then use Grep to locate the route handler and its imports, and read files incrementally along the call chain. B) Use Bash to run a full-text word count across the repository and read the files with the highest counts, on the assumption larger files hold core business logic. C) Use Read to open every file under the src directory up front, building a complete mental model of the whole codebase before looking for the login flow specifically.

D) Use Glob to list every file in the repository sorted by modification time, then read the twenty most recently modified files on the assumption they relate to login.

2. 구간별 직독직해 번역

QUESTION:

A newly onboarded architect 새롭게 온보딩된 아키텍트가
asks Claude Code to trace Claude Code에 추적하도록 요청합니다
how a login request flows 로그인 요청이 어떻게 흐르는지
from the HTTP route handler HTTP 라우트 핸들러로부터
through to the database call, 데이터베이스 호출까지
in a codebase 코드베이스 내에서
Claude has not explored yet. Claude가 아직 탐색하지 않은
To build this understanding efficiently 이러한 이해를 효율적으로 구축하기 위해
while keeping context usage low, 컨텍스트 사용량을 낮게 유지하면서
what is the best incremental strategy? 가장 좋은 점진적 전략은 무엇인가요?

OPTIONS:

Option A: Start by reading CLAUDE.md or AGENTS.md CLAUDE.md 또는 AGENTS.md를 읽는 것으로 시작하여

if they exist 존재하는 경우

to gain high-level architecture context, 고위급 아키텍처 컨텍스트를 얻고

then use Grep to locate 그런 다음 Grep을 사용하여 찾고

the route handler and its imports, 라우트 핸들러와 해당 import들을

and read files incrementally 파일들을 점진적으로 읽는 것

along the call chain. 호출 체인을 따라

Option B: Use Bash to run a full-text word count Bash를 사용하여 전체 텍스트 단어 수를 실행하고

across the repository 저장소 전체에 걸쳐

and read the files 파일들을 읽는 것

with the highest counts, 가장 높은 단어 수를 가진

on the assumption 가정 하에

larger files hold core business logic. 더 큰 파일이 핵심 비즈니스 로직을 가지고 있다는

Option C: Use Read to open every file Read를 사용하여 모든 파일을 열고

under the src directory up front, src 디렉토리 아래의 파일들을 사전에 미리

building a complete mental model 완벽한 멘탈 모델을 구축하는 것

of the whole codebase 전체 코드베이스에 대한

before looking for the login flow specifically. 구체적으로 로그인 흐름을 찾기 전에

Option D: Use Glob to list every file Glob을 사용하여 모든 파일 목록을 조회하고

in the repository 저장소 내의

sorted by modification time, 수정 시간순으로 정렬하여

then read the twenty most recently modified files 최근에 수정된 20개 파일을 읽는 것

on the assumption 가정 하에

they relate to login. 그 파일들이 로그인과 관련이 있다는

3. 정답 및 해설 (Answer & Explanation)

정답: A번: Start by reading CLAUDE.md or AGENTS.md if they exist to gain high-level architecture context, then use Grep to locate the route handler and its imports, and read files incrementally along the call chain.

정답 및 해설: 핵심 개념: 대규모 코드베이스에서 LLM 에이전트(Claude Code 등)를 사용할 때 전체 코드를 한꺼번에 토큰화하여 컨텍스트에 담는 것은 토큰 비용 및 환각(Hallucination) 위험을 높입니다. 따라서 프로젝트 설명 파일로 전체 아키텍처를 파악한 뒤, 검색 도구로 시작 지점을 찾아 호출 체인을 따라 점진적으로 관련 파일만 읽어 들여 컨텍스트 소모를 최소화해야 합니다.

문제 상황 분석:

- 새로운 아키텍트가 아직 Claude가 읽어보지 않은 프로젝트에서 로그인 흐름 추적을 요청함.
- 목적은 코드베이스를 효율적으로 이해하면서 토큰 사용량을 최소한으로 유지하는 것임.
- 무작위 파일 열람이나 전체 파일 일괄 로딩을 피하고 필요한 경로만 점진적으로 파악하는 전략이 필요함.

A번이 정답인 이유:

- CLAUDE.md나 AGENTS.md 같은 프로젝트 설정 문서를 먼저 읽어 전체적인 아키텍처 구조 및 가이드라인을 효율적으로 파악합니다.
- Grep 도구를 활용해 로그인 관련 라우트 핸들러 위치를 검색하여 탐색의 시작점을 지정합니다.
- 라우트 핸들러부터 DB 호출까지 연결된 import 및 함수 호출 체인을 따라 필요한 파일만 순차적으로 읽어 들임으로써 불필요한 토큰 사용을 완벽히 방지합니다.

오답 분석:

- Option B (오답): 파일의 크기가 크다고 해서 핵심 비즈니스 로직이 담겨 있다고 보장할 수 없으며 불필요한 대용량 파일로 컨텍스트가 낭비됩니다.
- Option C (오답): src 디렉토리 하위의 모든 파일을 한꺼번에 열어 읽는 방식은 컨텍스트 윈도우 한계를 초과하거나 대량의 토큰 소모를 야기하므로 조건에 위배됩니다.
- Option D (오답): 최근 수정된 파일이 로그인 로직과 관련이 있다는 보장이 없으며 무작위 파일 읽기는 비효율적입니다.

62번 문제

1. 문제 원문

A subagent handling document translation calls a `translate_text` tool that fails with a connection timeout on the first attempt. The subagent's local retry policy allows up to 2 automatic retries for transient errors before escalating. The second retry also times out. What should the subagent do next?

A) Silently return a fabricated translation to the coordinator, using a cached fallback response from a prior successful call to avoid workflow interruption, despite the `translate_text` tool timeouts. B) Stop retrying and propagate the failure to the coordinator, noting the `errorCategory`, that timeouts persisted across the allowed local attempts, and what text remained untranslated. C) Continue retrying indefinitely at the subagent level, logging each

attempt and resetting the retry count, ensuring that transient errors like timeouts are never escalated to the coordinator. D) Immediately reclassify the error as a permission failure and propagate it to the coordinator, including the permission flag and that repeated timeouts indicate an access-control issue, while aborting further retries.

2. 구간별 직독직해 번역

QUESTION:

A subagent 서브에이전트가

handling document translation 문서 번역을 처리하는

calls a `translate_text` tool `translate_text` 도구를 호출합니다

that fails with a `connection timeout` 연결 시간 초과로 실패하는

on the first attempt. 첫 번째 시도에서.

The subagent's local retry policy 서브에이전트의 로컬 재시도 정책은

allows up to 2 automatic retries 최대 2회의 자동 재시도를 허용합니다

for transient errors 일시적인 오류에 대해

before escalating. 상위(코디네이터)로 상격하기 전에.

The second retry also times out. 두 번째 재시도 역시 시간 초과됩니다.

What should the subagent do next? 서브에이전트는 다음에 무엇을 해야 합니까?

OPTIONS:

Option A: Silently return a `fabricated translation` 조용히 조작된 번역 결과를 반환합니다

to the coordinator, 코디네이터에게,

using a `cached fallback response` 캐시된 폴백 응답을 사용하여

from a prior successful call 이전의 성공적인 호출로부터 얻은

to avoid workflow interruption, 워크플로우 중단을 피하기 위해,

despite the `translate_text tool timeouts`. `translate_text` 도구 시간 초과에도 불구하고.

Option B: Stop retrying 재시도를 중지하고

and propagate the failure to the coordinator, 실패를 코디네이터에게 전파합니다,
noting the errorCategory, errorCategory를 기록하고,
that timeouts persisted 시간 초과가 지속되었음을 (기록하며)
across the allowed local attempts, 허용된 로컬 시도 횟수 동안,
and what text remained untranslated. 그리고 어떤 텍스트가 번역되지 않은 채로 남았는지를.

Option C: Continue retrying indefinitely 무한히 재시도를 계속합니다

at the subagent level, 서브에이전트 수준에서,
logging each attempt 각 시도를 로깅하고
and resetting the retry count, 재시도 횟수를 리셋하면서,
ensuring that transient errors 일시적인 오류가 (보장하도록)
like timeouts 시간 초과와 같은
are never escalated to the coordinator. 코디네이터에게 절대 상격되지 않도록.

Option D: Immediately reclassify the error 오류를 즉시 재분류합니다

as a permission failure 권한 실패로
and propagate it to the coordinator, 그리고 그것을 코디네이터에게 전파합니다,
including the permission flag 권한 플래그와
and that repeated timeouts indicate 반복된 시간 초과가 의미한다는 것을 포함하여
an access-control issue, 접근 제어 문제임을,
while aborting further retries. 추가 재시도를 중단하는 동안.

3. 정답 및 해설 (Answer & Explanation)

정답: B번: Stop retrying and propagate the failure to the coordinator, noting the errorCategory, that timeouts persisted across the allowed local attempts, and what text remained untranslated.

정답 및 해설: 핵심 개념: 계층적 AI 에이전트 아키텍처에서 서브에이전트는 허용된 로컬 재시도 횟수를 초과할 경우, 작업을 즉시 중단하고 상세한 상태 정보를 상위 조율자에게 전달해야 합니다.

문제 상황 분석:

- 첫 번째 시도 실패 후, 정책에 따라 허용된 2회의 자동 재시도를 수행함.
- 두 번째 재시도까지 모두 시간 초과되어 로컬에서 허용된 재시도 기회를 모두 소모함.
- 로컬 재시도로 해결되지 않는 지속적 오류 상태이므로 에스컬레이션 단계에 도달함.

B번이 정답인
